Introduction aux Bases de Données Relationnelles et Requêtes SQL de Base

Thème principal : Bases de Données Relationnelles, Modélisation, Clés Primaires/Étrangères et Requêtes SQL (SELECT, WHERE, JOIN)

Synthèse exécutive : Ce cours pose les fondations fondamentales de la ressource R1.05 (Introduction aux Bases de Données). L'objectif est de comprendre pourquoi nous utilisons des bases de données relationnelles plutôt que de simples fichiers texte ou des tableurs, comment structurer l'information sans redondance via les clés primaires et clés étrangères, et comment interroger efficacement ces données grâce au langage SQL (requêtes de sélection, filtrage et jointures internes).

1. Le Pourquoi : Des Tableurs aux Bases de Données Relationnelles

Dans les applications informatiques réelles, stocker des informations dans un fichier Excel ou un tableau plat pose rapidement des problèmes majeurs : si un client change d'adresse et que cette adresse est répétée 50 fois pour chacune de ses commandes, il faut la modifier 50 fois au risque d'oublier des lignes et de créer des incohérences. Le modèle relationnel résout ce problème en séparant les entités distinctes (par exemple, les 'Clients' d'un côté et les 'Commandes' de l'autre) dans des tables séparées et en établissant des liens stricts entre elles via des clés.

- Inconvénients des tableurs : Redondance de données, anomalies de mise à jour, risque d'incohérence.
- SGBD (Système de Gestion de Bases de Données) : Logiciel garantissant l'intégrité, la sécurité et l'accès concurrentiel aux données.
- Modèle Relationnel : Organisation des données sous forme de tables reliées logiquement entre elles.

Cas concret enseignant : *Le professeur a illustré cela avec l'analogie du système bancaire : si un compte en banque stockait le nom du titulaire et son adresse sur chaque ligne de transaction bancaire, un simple déménagement nécessiterait de repasser sur des milliers d'opérations passées, augmentant le risque d'erreur de saisie.*

2. Clés Primaires et Clés Étrangères : L'Ancre de la Cohérence

Chaque n-uplet (ligne) d'une table doit être unique. Pour cela, on définit une Clé Primaire (ex: id_etudiant). Celle-ci ne peut jamais être nulle (NOT NULL) ni dupliquée. Lorsque nous voulons associer une entité à une autre (par exemple lier un étudiant à son groupe de TD), nous insérons la clé primaire du groupe de TD en tant que Clé Étrangère dans la table Étudiant. Le SGBD vérifie automatiquement que l'identifiant du groupe existe bel et bien avant d'autoriser l'insertion.

- Clé Primaire (Primary Key - PK) : Attribut (ou ensemble d'attributs) identifiant de façon unique chaque enregistrement d'une table.
- Clé Étrangère (Foreign Key - FK) : Attribut dans une table qui fait référence à la clé primaire d'une autre table.
- Contrainte d'Intégrité Référentielle : Règle empêchant de pointer vers une clé primaire inexistante.

Cas concret enseignant : *Exemple du numéro d'Insee / Sécurité Sociale : Même si deux personnes partagent exactement le même nom et le même prénom à Bayonne, leur numéro d'identifiant unique empêche toute confusion au niveau de la base de données de l'IUT.*

3. Extraction de Données avec SQL : SELECT, WHERE et INNER JOIN

Le langage SQL (Structured Query Language) est un langage déclaratif. On lui indique CE QUE l'on souhaite obtenir et non COMMENT l'obtenir. La structure minimale d'une requête de recherche est SELECT colonnes FROM table WHERE condition. Pour croiser des informations provenant de deux tables distinctes, on utilise INNER JOIN table2 ON table1.cle_fk = table2.cle_pk.

• Projection (SELECT) : Choix des colonnes à afficher.

• Sélection/Filtrage (WHERE) : Explicitation des conditions logiques sur les lignes à conserver.

• Jointure (INNER JOIN ... ON) : Fusion logique de deux tables basée sur la correspondance entre clé étrangère et clé primaire.

Cas concret enseignant : *L'enseignant insiste : 'N'utilisez jamais la syntaxe implicite du produit cartésien avec une virgule dans le FROM (ex: FROM Etudiant, Groupe WHERE Etudiant.id_g = Groupe.id). Utilisez TOUJOURS la syntaxe explicite JOIN ... ON, car c'est la norme ISO moderne exigeante aux examens et en entreprise.'*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne doit-on pas utiliser SELECT * dans un code applicatif ou un examen ?
Réponse du professeur : Utiliser 'SELECT *' récupère toutes les colonnes de la table. En production, cela dégrade fortement les performances réseau et mémoire en chargeant des données inutiles. En examen, cela montre un manque de précision : il faut toujours projeter uniquement les attributs demandés dans l'énoncé.

Question étudiant : Que se passe-t-il si j'essaie de supprimer un groupe qui contient encore des étudiants ?
Réponse du professeur : Le SGBD bloque la suppression et renvoie une erreur de contrainte d'intégrité référentielle, sauf si la suppression en cascade (ON DELETE CASCADE) a été explicitement configurée à la création de la table.

■ Exercice & Démonstration d'Application

Énoncé : On considère deux tables : Etudiant(id_etudiant, nom, prenom, #id_groupe) et Groupe(id_groupe, nom_groupe, promo). Écrire la requête SQL permettant d'afficher le nom et le prénom des étudiants appartenant à la promo 'BUT1-INFO' du groupe nommé 'A1'.

Résolution pas à pas :
Étape 1 : Identifier les tables nécessaires -> Etudiant et Groupe. Étape 2 : Déterminer la condition de jointure -> Etudiant.id_groupe = Groupe.id_groupe. Étape 3 : Identifier les filtres WHERE -> Groupe.promo = 'BUT1-INFO' ET Groupe.nom_groupe = 'A1'. Étape 4 : Définir la projection SELECT -> Etudiant.nom, Etudiant.prenom. Requête SQL rédigée : SELECT e.nom, e.prenom FROM Etudiant e INNER JOIN Groupe g ON e.id_groupe = g.id_groupe WHERE g.promo = 'BUT1-INFO' AND g.nom_groupe = 'A1';

Résultat : La requête retourne la liste exacte des noms et prénoms des étudiants inscrits dans le groupe A1 de BUT1-INFO, sans aucune redondance et en respectant la norme SQL ANSI.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la propriété fondamentale d'une clé primaire ?

R : Elle doit être unique pour chaque ligne de la table et ne peut jamais contenir la valeur NULL.

Q : À quoi sert la clause INNER JOIN en SQL ?

R : Elle permet de combiner les lignes de deux tables lorsque la condition de jointure (égalité entre clé étrangère et clé primaire) est vérifiée.

Q : Pourquoi la syntaxe 'FROM TableA, TableB WHERE TableA.id = TableB.id_a' est-elle déconseillée ?

R : Parce qu'il s'agit d'une jointure implicite par produit cartésien, moins lisible, obsolète et sujette à des erreurs d'oubli de condition de jointure.